LEVEL 3 - SQL Functions

Aggregate, String and Numeric Functions
with Explanations, Examples and Practice Solutions

Contents

1. Sample Table Used in This Guide
2. Aggregate Functions: COUNT, SUM, AVG, MIN, MAX
3. String Functions: UPPER, LOWER, CONCAT, LENGTH, TRIM, REPLACE, SUBSTRING
4. Numeric Functions: ROUND, CEIL, FLOOR, ABS, MOD
5. Practice Questions 1-15 with Answers
6. Quick Revision Cheat Sheet

1. Sample Table Used in This Guide

A function is a ready-made SQL command that takes input values and returns a result. All examples use the table below. Note: syntax is mostly MySQL; small differences for other databases are mentioned where needed.

```
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    first_name VARCHAR(30),
    last_name  VARCHAR(30),
    course     VARCHAR(20),
    marks      INT
);

INSERT INTO students VALUES
(1,'Ravi','Kumar','BTech',85),
(2,'Sita','Devi','BTech',92),
(3,'Arjun','Rao','MBA',78),
(4,'Priya','Sharma','MBA',88),
(5,'Kiran','Reddy','BSc',67),
(6,'Anil','Varma','BSc',74);
```

student_id	first_name	last_name	course	marks
1	Ravi	Kumar	BTech	85
2	Sita	Devi	BTech	92
3	Arjun	Rao	MBA	78
4	Priya	Sharma	MBA	88
5	Kiran	Reddy	BSc	67
6	Anil	Varma	BSc	74

Types of SQL functions

Type	What it does	Rows returned
Aggregate	Works on many rows and returns ONE value (a summary)	One value per group
String	Works on text values (change, measure, cut, join)	One value per row
Numeric	Works on numbers (round, absolute, remainder)	One value per row

2. Aggregate Functions

Aggregate functions look at a set of rows and combine them into a single result. They are often used with **GROUP BY** to get one result per group. Important: aggregate functions ignore NULL values (except `COUNT(*)`).

2.1 COUNT()

Counts the number of rows. `COUNT(*)` counts all rows; `COUNT(column)` counts only rows where that column is NOT NULL; `COUNT(DISTINCT column)` counts unique values.

Syntax:

```
SELECT COUNT(*) FROM table_name;
SELECT COUNT(column) FROM table_name;
SELECT COUNT(DISTINCT column) FROM table_name;
```

Example:

```
SELECT COUNT(*) AS total_students FROM students;
SELECT COUNT(DISTINCT course) AS total_courses FROM students;

Output:
total_students = 6
total_courses  = 3
```

2.2 SUM()

Adds up all values of a numeric column. NULL values are ignored.

Syntax:

```
SELECT SUM(column) FROM table_name;
```

Example:

```
SELECT SUM(marks) AS total_marks FROM students;

Output:
total_marks = 484      (85+92+78+88+67+74)
```

2.3 AVG()

Returns the average (mean) of a numeric column = SUM / number of non-NULL values.

Syntax:

```
SELECT AVG(column) FROM table_name;
```

Example:

```
SELECT AVG(marks) AS average_marks FROM students;

Output:
average_marks = 80.6667      (484 / 6)
```

2.4 MIN()

Returns the smallest value in a column. Works on numbers, text (alphabetical) and dates.

Syntax:

```
SELECT MIN(column) FROM table_name;
```

Example:

```
SELECT MIN(marks) AS lowest FROM students;
SELECT MIN(first_name) FROM students;
```

```
Output:
lowest = 67
Anil              (first alphabetically)
```

2.5 MAX()

Returns the largest value in a column. Works on numbers, text and dates.

Syntax:

```
SELECT MAX(column) FROM table_name;
```

Example:

```
SELECT MAX(marks) AS highest FROM students;
```

```
Output:
highest = 92
```

3. String Functions

String functions work on text. They return a changed or measured version of each value. They do not modify the stored data unless you use UPDATE.

3.1 UPPER()

Converts all letters of a string to UPPERCASE.

Syntax:

```
SELECT UPPER(column) FROM table_name;
```

Example:

```
SELECT UPPER(first_name) AS name_upper FROM students;
```

```
Output:
RAVI, SITA, ARJUN, PRIYA, KIRAN, ANIL
```

3.2 LOWER()

Converts all letters of a string to lowercase.

Syntax:

```
SELECT LOWER(column) FROM table_name;
```

Example:

```
SELECT LOWER(first_name) AS name_lower FROM students;
```

```
Output:
ravi, sita, arjun, priya, kiran, anil
```

3.3 CONCAT()

Joins two or more strings into one. If any argument is NULL, MySQL returns NULL (use CONCAT_WS or COALESCE to handle this). In Oracle/PostgreSQL you can also use ||.

Syntax:

```
SELECT CONCAT(string1, string2, ...) FROM table_name;
```

Example:

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name FROM students;
```

```
Output:
Ravi Kumar, Sita Devi, Arjun Rao, Priya Sharma, Kiran Reddy, Anil Varma
```

3.4 LENGTH()

Returns the number of characters in a string (MySQL: LENGTH counts bytes, CHAR_LENGTH counts characters; SQL Server uses LEN()).

Syntax:

```
SELECT LENGTH(column) FROM table_name;
```

Example:

```
SELECT first_name, LENGTH(first_name) AS name_length FROM students;
```

```
Output:
Ravi  4
Sita  4
Arjun 5
Priya 5
Kiran 5
Anil  4
```

3.5 TRIM()

Removes extra spaces from the start and end of a string. Variants: LTRIM() removes left spaces, RTRIM() removes right spaces.

Syntax:

```
SELECT TRIM(column) FROM table_name;
SELECT TRIM(BOTH 'x' FROM column) FROM table_name;  -- remove a character
```

Example:

```
SELECT TRIM('   Hello SQL   ') AS cleaned;
SELECT LENGTH(TRIM('   Hello SQL   ')) AS len;
```

```
Output:
cleaned = Hello SQL
len     = 9
```

3.6 REPLACE()

Replaces every occurrence of a substring with another substring. Case-sensitive.

Syntax:

```
SELECT REPLACE(column, 'old_text', 'new_text') FROM table_name;
```

Example:

```
SELECT REPLACE(course, 'BTech', 'B.Tech') AS new_course FROM students;
SELECT REPLACE('Hello World', 'World', 'SQL');
```

```
Output:
B.Tech, B.Tech, MBA, MBA, BSc, BSc
Hello SQL
```

3.7 SUBSTRING()

Extracts part of a string. Positions start at 1 (not 0). SUBSTR() works in Oracle; SUBSTRING and SUBSTR both work in MySQL.

Syntax:

```
SELECT SUBSTRING(column, start_position, length) FROM table_name;
```

Example:

```
SELECT SUBSTRING(first_name, 1, 3) AS short_name FROM students;
SELECT SUBSTRING('DATABASE', 5);
```

```
Output:
Rav, Sit, Arj, Pri, Kir, Ani
BASE        (from position 5 to the end)
```

4. Numeric Functions

Numeric functions work on numbers and return one result for each row (or each value you give).

4.1 ROUND()

Rounds a number to the given number of decimal places. Use 0 (or omit) to round to a whole number. A negative value rounds to the left of the decimal point.

Syntax:

```
SELECT ROUND(number, decimal_places);
```

Example:

```
SELECT ROUND(80.6667, 2);
SELECT ROUND(80.6667, 0);
SELECT ROUND(1234.5, -2);
```

```
Output:
80.67
81
1200
```

4.2 CEIL() / CEILING()

Rounds a number UP to the next whole number (the smallest integer greater than or equal to it). SQL Server uses CEILING().

Syntax:

```
SELECT CEIL(number);
```

Example:

```
SELECT CEIL(80.2);
SELECT CEIL(-4.7);
```

```
Output:
81
-4
```

4.3 FLOOR()

Rounds a number DOWN to the previous whole number (the largest integer less than or equal to it).

Syntax:

```
SELECT FLOOR(number);
```

Example:

```
SELECT FLOOR(80.9);
SELECT FLOOR(-4.2);
```

```
Output:
80
-5
```

4.4 ABS()

Returns the absolute value, i.e. the number without its negative sign (the distance from zero).

Syntax:

```
SELECT ABS(number);
```

Example:

```
SELECT ABS(-25);
SELECT ABS(marks - 80) AS difference_from_80 FROM students;

Output:
25
5, 12, 2, 8, 13, 6
```

4.5 MOD()

Returns the remainder after dividing one number by another. You can also use the % operator in MySQL, SQL Server and PostgreSQL. Common use: check even/odd.

Syntax:

```
SELECT MOD(number, divisor);
```

Example:

```
SELECT MOD(10, 3);
SELECT student_id, MOD(student_id, 2) AS rem FROM students;  -- 0 = even, 1 = odd

Output:
1
IDs 1,3,5 -> 1 (odd) ; IDs 2,4,6 -> 0 (even)
```

CEIL vs FLOOR vs ROUND (quick comparison)

Value	ROUND(x)	CEIL(x)	FLOOR(x)
4.2	4	5	4
4.5	5	5	4
4.8	5	5	4
-4.2	-4	-4	-5

5. Practice Questions with Answers

Using the **students** table from Section 1.

Q1. Count total students.

```
SELECT COUNT(*) AS total_students
FROM students;
```

```
Output:
total_students
6
```

Q2. Calculate average marks.

```
SELECT AVG(marks) AS average_marks
FROM students;
```

```
Output:
average_marks
80.6667
```

Q3. Find highest marks.

```
SELECT MAX(marks) AS highest_marks
FROM students;
```

```
Output:
highest_marks
92
```

Q4. Find lowest marks.

```
SELECT MIN(marks) AS lowest_marks
FROM students;
```

```
Output:
lowest_marks
67
```

Q5. Calculate total marks.

```
SELECT SUM(marks) AS total_marks
FROM students;
```

```
Output:
total_marks
484
```

Q6. Convert names to uppercase.

```
SELECT UPPER(first_name) AS name_upper
FROM students;
```

```
Output:
RAVI
SITA
ARJUN
PRIYA
KIRAN
ANIL
```

Q7. Convert names to lowercase.

```
SELECT LOWER(first_name) AS name_lower
FROM students;
```

```
Output:
ravi
sita
arjun
priya
kiran
anil
```

Q8. Find name length.

```
SELECT first_name, LENGTH(first_name) AS name_length
FROM students;
```

```
Output:
first_name | name_length
Ravi       | 4
Sita       | 4
Arjun      | 5
Priya      | 5
Kiran      | 5
Anil       | 4
```

Q9. Combine first and last names.

```
SELECT CONCAT(first_name, ' ', last_name) AS full_name
FROM students;
```

```
Output:
Ravi Kumar
Sita Devi
Arjun Rao
Priya Sharma
Kiran Reddy
Anil Varma
```

Q10. Round average marks.

```
SELECT ROUND(AVG(marks), 2) AS avg_2_decimals,
       ROUND(AVG(marks))    AS avg_whole
FROM students;
```

```
Output:
avg_2_decimals | avg_whole
80.67          | 81
```

Q11. Calculate difference between maximum and minimum marks.

```
SELECT MAX(marks) - MIN(marks) AS marks_range
FROM students;
```

```
Output:
marks_range
25      (92 - 67)
```

Q12. Count students by course.

```
SELECT course, COUNT(*) AS total_students
FROM students
GROUP BY course;
```

```
Output:
course | total_students
BTech  | 2
MBA    | 2
BSc    | 2
```

Q13. Calculate average marks by course.

```
SELECT course, AVG(marks) AS avg_marks
FROM students
GROUP BY course;
```

```
Output:
course | avg_marks
BTech  | 88.5
MBA    | 83.0
BSc    | 70.5
```

Q14. Find maximum marks by course.

```
SELECT course, MAX(marks) AS max_marks
FROM students
GROUP BY course;
```

```
Output:
course | max_marks
BTech  | 92
MBA    | 88
BSc    | 74
```

Q15. Find minimum marks by course.

```
SELECT course, MIN(marks) AS min_marks
FROM students
GROUP BY course;
```

```
Output:
course | min_marks
BTech  | 85
MBA    | 78
BSc    | 67
```

How GROUP BY works (Q12-Q15)

GROUP BY splits the rows into groups that share the same value (here, the same course). The aggregate function is then applied to each group separately. Rule: every column in SELECT that is not inside an aggregate function must appear in GROUP BY.

6. Quick Revision Cheat Sheet

Function	Purpose	Example	Result
COUNT()	Number of rows	COUNT(*)	6
SUM()	Total of values	SUM(marks)	484
AVG()	Average	AVG(marks)	80.6667
MIN()	Smallest value	MIN(marks)	67
MAX()	Largest value	MAX(marks)	92
UPPER()	To uppercase	UPPER('sql')	SQL
LOWER()	To lowercase	LOWER('SQL')	sql
CONCAT()	Join strings	CONCAT('a','b')	ab
LENGTH()	Count characters	LENGTH('Ravi')	4
TRIM()	Remove edge spaces	TRIM(' hi ')	hi
REPLACE()	Swap text	REPLACE('cat','c','b')	bat
SUBSTRING()	Cut part of text	SUBSTRING('Hello',1,3)	Hel
ROUND()	Round number	ROUND(3.14159,2)	3.14
CEIL()	Round up	CEIL(3.1)	4
FLOOR()	Round down	FLOOR(3.9)	3
ABS()	Remove negative sign	ABS(-7)	7
MOD()	Remainder	MOD(10,3)	1

Key points to remember

- Aggregate functions return ONE value per group and ignore NULLs (except COUNT(*)).
- Use GROUP BY to apply aggregate functions to each category separately.
- WHERE filters rows BEFORE grouping; HAVING filters groups AFTER aggregation.
- String positions in SUBSTRING start at 1.
- Use AS to give a readable name (alias) to any function result.
- Function names and syntax can vary slightly between MySQL, SQL Server, Oracle and PostgreSQL.